

Algorytmy i struktury danych

Zajęcia laboratoryjne – 11 czerwca

Grafy – ogólne informacje

Poprzez *graf* rozumiemy strukturę składającą się z wierzchołków oraz krawędzi między ich parami. Formalnie, graf $G = (V, E)$ to para zbiorów: V to zbiór wierzchołków (*vertices*), E to zbiór krawędzi (*edges*).

- W **grafie nieskierowanym** krawędzie to dwuelementowe podzbiory zbioru wierzchołków:

$$E \subset \{\{u, v\} \mid u, v \in V\}.$$

Krawędzie łączą wierzchołki; kierunek połączenia nie jest istotny, ponieważ $\{u, v\} = \{v, u\}$ – krawędzie są jednoznacznie zdefiniowane poprzez wskazanie wierzchołków, które mają łączyć.

- W **grafie skierowanym** jest inaczej – krawędź $u \rightarrow v$ jest inna niż krawędź $v \rightarrow u$. Formalnie:

$$E \subset \{(u, v) \mid u, v \in V\},$$

gdzie notacja (u, v) (w przeciwieństwie do $\{u, v\}$) oznacza skierowaną parę; $(u, v) \neq (v, u)$.

- Zazwyczaj nie rozważamy krawędzi łączących wierzchołki same ze sobą (tzn. $\{u, u\}$ lub $u \rightarrow u / (u, u)$).

Graf nieskierowany:	A	Graf skierowany:	A
$V = \{A, B, C, D, E\}$	$\begin{array}{c} / \backslash \\ B \text{ --- } C \end{array}$	$V = \{A, B, C, D, E\}$	$\begin{array}{c} \swarrow \searrow \\ B \rightarrow C \end{array}$
$E = \{\{A, B\}, \{A, C\}, \{B, C\}, \{B, D\}, \{C, E\}, \{D, E\}\}$	$\begin{array}{cc} & \\ D \text{ --- } E \end{array}$	$E = \{(A, B), (A, C), (B, C), (B, D), (C, E), (D, E)\}$	$\begin{array}{cc} \downarrow & \downarrow \\ D \rightarrow E \end{array}$

Istotne jest, żeby pamiętać, że nazwy wierzchołków to jedynie etykiety, które pozwalają na ich rozróżnienie. Nie ma znaczenia, czy wierzchołki nazwiemy $\{A, B, C\}$ czy $\{1, 2, 3\}$, o ile krawędzie łączą je w identyczny sposób. Na przykład:

- Graf G o wierzchołkach $\{A, B, C\}$ i pojedynczej krawędzi $\{A, B\}$ jest tym samym grafem (formalnie: jest izomorficzny) z grafem G^* o wierzchołkach $\{A, B, C\}$ i krawędzi $\{B, C\}$ – wystarczy zmienić (dokonać permutacji) etykiet wierzchołków: $A \mapsto B, B \mapsto C, C \mapsto A$.
- Graf G jest również tożsamy z grafem o wierzchołkach $\{X, Y, Z\}$ i krawędzi $\{X, Y\}$ poprzez podmianę etykiet wierzchołków: $A \mapsto X, Y \mapsto B, C \mapsto Z$.

Reprezentacje grafów w komputerze

Założmy, że chcemy zareprezentować pewien graf o N wierzchołkach. Jest na to kilka sposobów.

- (1) **Lista krawędzi:** tworzymy listę, gdzie każdy element to krawędź (czyli lista zawierająca dwa wierzchołki):

$$[[A, B], [A, C], [A, E], [B, C], [C, E], [D, E]].$$

Taka reprezentacja jest prosta, łatwo wczytać ją z pliku, jest wygodna w przypadku algorytmów, które operują głównie na krawędziach (np. algorytm Kruskala). Sprawdza się gorzej, gdy potrzebujemy mieć możliwość szybkiego sprawdzenia sąsiadów danego wierzchołka. Na przykład, jeśli chcemy znaleźć wszystkich sąsiadów wierzchołka E w reprezentacji powyżej, musimy przeszukać wszystkie krawędzie w liście.

- (2) **Macierz sąsiedztwa:** tworzymy macierz kwadratową $N \times N$, gdzie każdy wiersz i kolumna odpowiadają jednemu wierzchołkowi. Wartości wypełniamy według zasady:

$$M_{ij} = \begin{cases} 1 & \text{jeżeli istnieje krawędź z } i \text{ do } j, \\ 0 & \text{w przeciwnym przypadku.} \end{cases}$$

Dla przykładu, taka macierz może wyglądać następująco:

$$M = \begin{matrix} & \begin{matrix} A & B & C & D & E \end{matrix} \\ \begin{matrix} A \\ B \\ C \\ D \\ E \end{matrix} & \begin{bmatrix} 0 & 1 & 1 & 0 & 0 \\ 1 & 0 & 1 & 1 & 0 \\ 1 & 1 & 0 & 0 & 1 \\ 0 & 1 & 0 & 0 & 1 \\ 0 & 0 & 1 & 1 & 0 \end{bmatrix} \end{matrix}$$

Dla grafów nieskierowanych macierz jest symetryczna: $M_{ij} = M_{ji}$, ponieważ jeśli i jest sąsiadem j , to j jest sąsiadem i (co nie jest prawdą w przypadku grafu skierowanego).

Macierz sąsiedztwa jest wygodna i prosta koncepcyjnie, ale kosztuje $O(n^2)$ pamięci.

(3) Lista sąsiedztwa: dla każdego wierzchołka przechowujemy listę jego sąsiadów, na przykład:

A: [B, C, E]; B: [A, C, D]; C: [A, B, E]; D: [B, E]; E: [C, D]

W przypadku grafów skierowanych wypisujemy jedynie wierzchołki wychodzące.

Bardzo użyteczna – pozwala na iterację po sąsiadach danego wierzchołka, co jest powszechnym krokiem w wielu algorytmach (w tym w algorytmach przeszukiwania grafów: BFS i DFS).

(4) Macierz incydencji: wiersze to wierzchołki, kolumny to krawędzie – w komórkach zapisujemy, czy wierzchołek należy do krawędzi. Używana w (matematycznej) teorii grafów, mniej wygodna przy implementacji algorytmów.

Zadania

1. Dla obu grafów (nieskierowanego i skierowanego) przedstawionych w sekcji **Grafy – informacje ogólne** zapisz ich reprezentacje jako macierz sąsiedztwa i lista sąsiedztwa. Jaki jest stopień (w przypadku grafu skierowanego rozważ stopień wchodzący i wychodzący) każdego wierzchołka?
2. Napisz funkcję `build_adj_list(vertices: list, edges: list, directed: bool) -> dict`, która przyjmuje listę wierzchołków (ich etykiet), listę krawędzi (par etykiet) oraz wartość logiczną opisującą, czy graf jest skierowany czy nie – i zwraca słownik, gdzie klucze to etykiety wierzchołków, a wartości to listy sąsiadów.
3. Napisz funkcję `adj_matrix_from_adj_list(adj_list: dict) -> list`, która przyjmuje listę sąsiedztwa (w postaci, w jakiej tworzy ją funkcja z poprzedniego zadania), a zwraca dwuwymiarową tablicę `list[list[int]]` reprezentującą jej macierz sąsiedztwa.
4. W Pythonie zaimplementuj klasę `Graph`, która reprezentuje nieskierowany graf przy pomocy listy sąsiedztwa. Klasa powinna zawierać następujące funkcjonalności:
 - dodawanie wierzchołków – `add_vertex(self, v)`
 - dodawanie krawędzi – `add_edge(self, u, v)`
 - podgląd wszystkich wierzchołków – `show_vertices(self)`
 - podgląd wszystkich krawędzi – `show_edges(self)`
 - podgląd listy sąsiadów – `show_neighbors(self, v)`
 - sprawdzenie, czy dwa wierzchołki są połączone krawędzią – `has_edge(self, u, v)`
5. Zaprojektuj i z koduj funkcję `find_path(G: Graph, u, v)`, która przyjmuje obiekt klasy `Graph` (z poprzedniego zadania) oraz wierzchołki u, v i zwraca listę krawędzi, które należy po kolei przejść, żeby dostać się z u do v .
 - Możesz założyć, że graf jest spójny – czyli że taka ścieżka na pewno istnieje.
 - Algorytm może być naiwny, nie musi być szybki – w zadaniu chodzi przede wszystkim o własny pomysł.
 - Pomocne prawie na pewno mogą okazać się struktury stosu lub kolejki.